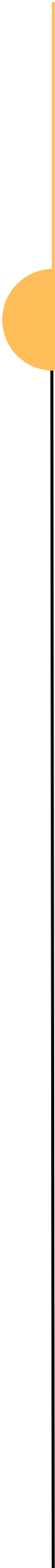


**GEETANJALI
DHOKE**

BUSINESS ANALYST



Technical Requirements Document (TRD)

Document Information

Field	Value
Project Name	
Document Name	Technical Requirements Document
Version	1
Author	
Reviewer	
Approved By	
Date	
Status	Draft / Review /Approved

Revision History

Version	Date	Author	Changes
1			Initial Draft

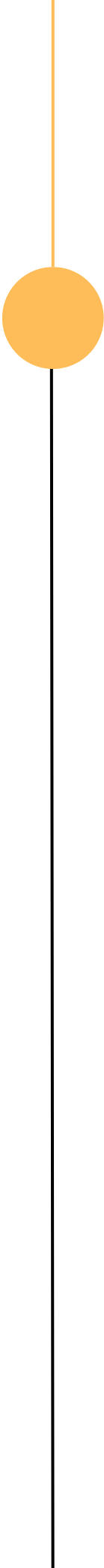


Table of Contents

1. Introduction
2. Technical Overview
3. System Architecture
4. Technology Stack
5. Infrastructure
6. Database Design
7. API Design
8. Authentication & Authorization
9. Business Logic
10. Third-Party Integrations
11. Performance & Scalability
12. Security
13. Logging & Monitoring
14. Error Handling
15. Deployment Strategy
16. Testing Strategy
17. Risks & Assumptions
18. Appendix

1. Introduction

Purpose

Describe the technical approach for implementing the software.

Example:

This document defines the architecture, technology stack, APIs, database, infrastructure, deployment, and implementation details for the Clinic Management System.

Scope

This document includes:

- Backend Architecture
- Frontend Architecture
- APIs
- Database
- Security
- Deployment
- Monitoring

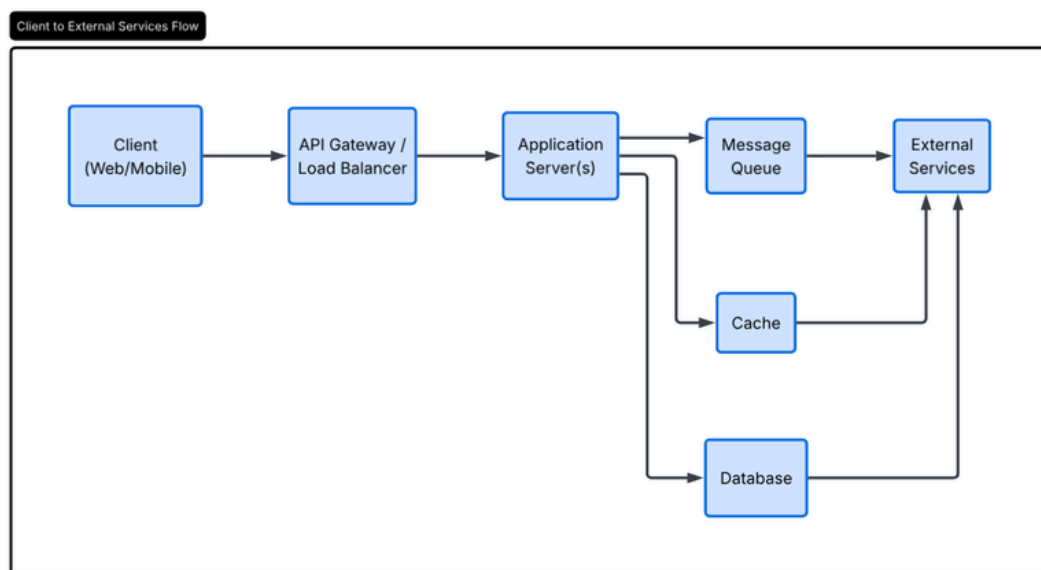
2. Technical Overview

Architecture Style

- Monolithic
- Microservices
- Serverless
- Event Driven

Describe why this architecture was selected.

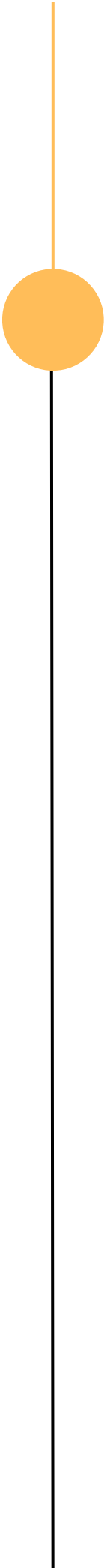
High-Level Architecture



3. Technology Stack

Frontend

Technology	Version
React	
Next.js	
Tailwind CSS	
TypeScript	



Backend

Technology	Version
Node.js	
Express/NestJS	
TypeScript	

Database

Technology	Purpose
PostgreSQL	Primary Database
Redis	Cache
MongoDB	Optional

Infrastructure

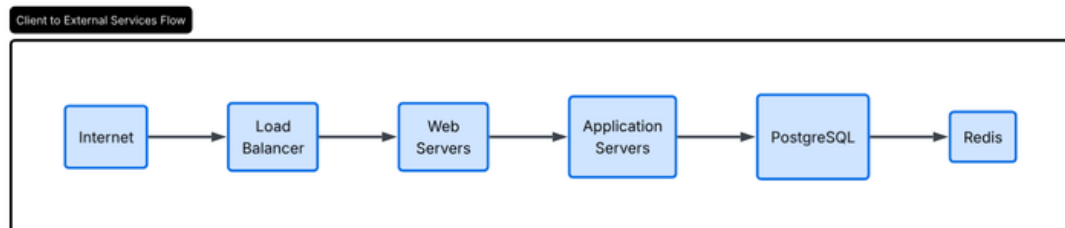
Tool	Purpose
Docker	Containerization
Kubernetes	Orchestration
Nginx	Reverse Proxy
AWS/GCP/Azure	Cloud

4. Infrastructure Architecture

Environment

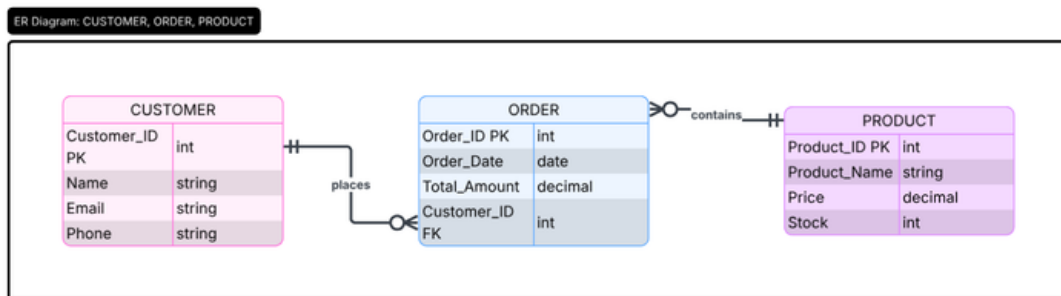
- Development
- QA
- Staging
- Production

Infrastructure Diagram



5. Database Design

Entity Relationship Diagram



Tables

Users

Column	Type	Constraints
id	UUID	Primary Key
name	VARCHAR	NOT NULL
email	VARCHAR	UNIQUE
password	TEXT	Encrypted

Patients

Column	Type
id	UUID
name	VARCHAR
dob	DATE

Relationships

- User → Appointments (One to Many)
- Doctor → Appointments (One to Many)
- Patient → Appointments (One to Many)

6. API Design

Authentication

POST /login

Request

```
{  
  "email": "",  
  "password": ""  
}
```

Success Response

```
{  
  "token": "",  
  "user": {}  
}
```

GET /patients

Headers

Authorization: Bearer Token

Response

```
[
  {
    "id": 1,
    "name": "John"
  }
]
```

API Standards

- REST
- JSON
- HTTP Status Codes
- Pagination
- Filtering
- Sorting

7. Authentication & Authorization

Authentication

- JWT
- OAuth
- Session

Authorization

RBAC

Role	Permissions
Admin	Full
Doctor	Patients
Receptionist	Appointments

8. Business Logic

Describe key workflows.

Example:

Appointment Booking

1. Validate patient.
2. Check doctor availability.
3. Reserve slot.
4. Save appointment.
5. Send notification.

9. Third-Party Integrations

Service	Purpose
Stripe/Razorpay	Payments
Twilio	SMS
WhatsApp API	Notifications
SendGrid	Emails

10. Performance Requirements

Response Time

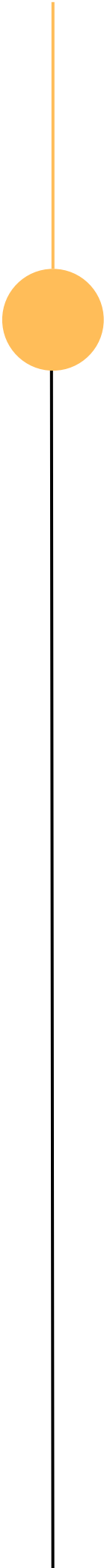
- API < 500 ms
- UI < 2 seconds

Scalability

- 10,000 concurrent users

Caching

- Redis
- CDN



11. Security

Authentication

JWT

Encryption

- HTTPS
- AES
- Password Hashing (bcrypt/Argon2)

Security Features

- Rate Limiting
- CSRF Protection
- XSS Protection
- SQL Injection Prevention
- CSP Headers

12. Logging

Application Logs

- Request
- Response
- Errors
- Authentication

Audit Logs

- Login
- Create
- Update
- Delete

13. Monitoring

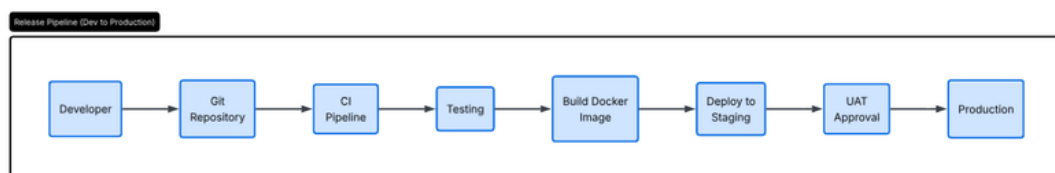
Tool	Purpose
Prometheus	Metrics
Grafana	Dashboards
ELK	Logs
Sentry	Error Tracking

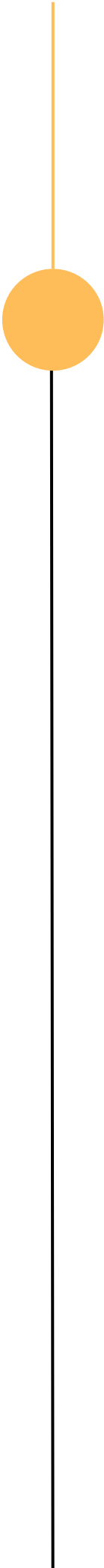
14. Error Handling

Error	Response
400	Validation Error
401	Unauthorized
403	Forbidden
404	Not Found
500	Internal Server Error

15. Deployment Strategy

CI/CD Pipeline





Deployment Steps

- 1. Build application.**
- 2. Run automated tests.**
- 3. Create Docker image.**
- 4. Push to registry.**
- 5. Deploy to Kubernetes.**
- 6. Verify health checks.**
- 7. Monitor logs.**

16. Backup & Recovery

- Daily database backup**
- Weekly full backup**
- Point-in-time recovery**
- Disaster recovery plan**

17. Testing Strategy

Unit Testing

Framework:

Integration Testing

API Testing

End-to-End Testing

Playwright/Cypress

Performance Testing

JMeter/k6

Security Testing

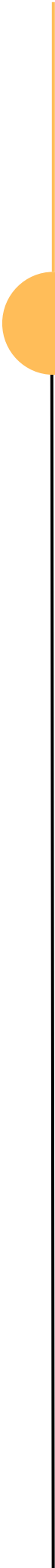
OWASP Top 10

18. Risks

Risk	Mitigation
Database failure	Replication
API downtime	Retry logic
High traffic	Auto scaling

19. Assumptions

- **Cloud infrastructure is available.**
- **Third-party APIs are operational.**
- **SSL certificates are configured.**



20. Appendix

Architecture Diagram

(Add diagrams)

Sequence Diagrams
(Add UML sequence diagrams)

Component Diagram
(Add UML component diagram)

Deployment Diagram
(Add deployment architecture)

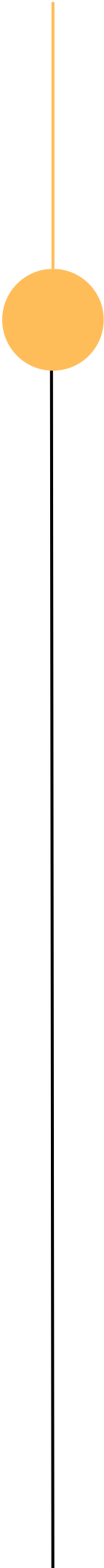
ER Diagram
(Add database ERD)

API Collection

- **OpenAPI (Swagger)**
- **Postman Collection**

Configuration
Example .env variables:

NODE_ENV=production
DATABASE_URL=
JWT_SECRET=
REDIS_URL=
API_BASE_URL=



Contact:

For more details or sample documents, feel free to connect with me:

Geetanjali Dhoke Business Analyst

 LinkedIn: <https://www.linkedin.com/in/geetanjali-dhoke-51a9711a1/>

 Email: gorgeousgeetanjali15@gmail.com

Thank you !